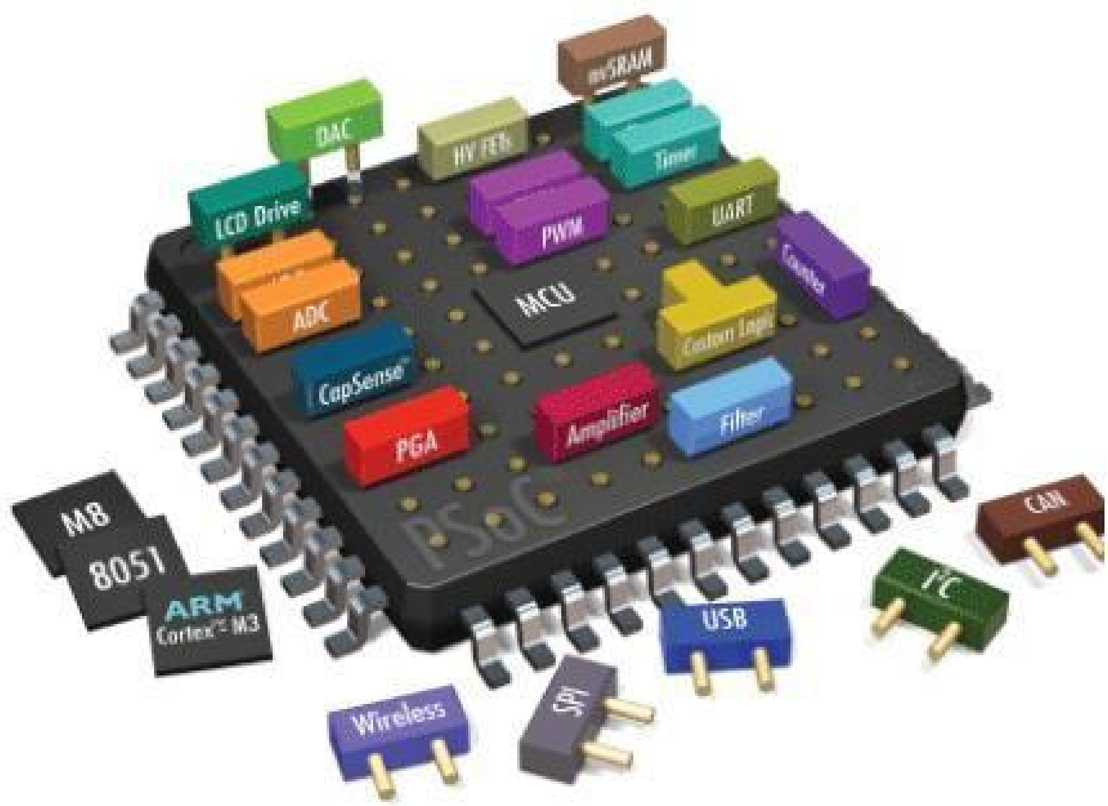
HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 01: Introduction to Edsim51 simulator with demo examples

Embedded Systems

Introduction to Edsim51 simulator with demo examples



Dated:

Week 01

Semester:

Fall 2023

Introduction to EdSim51 Simulator:

Simulator is a free simulator for the popular 8051 microcontrollers. In EDSIM51, a virtual 8051 is interfaced with virtual peripherals such as a keypad, motor, display, UART, etc. The student can write 8051 assembly code, step through the code and observe the effects each line has on the internal memory and the external peripherals. The following is a list of virtual peripherals in EDSIM51:

- Analogue-to-Digital Converter (ADC)
- Comparator
- UART
- 4 Multiplexed 7-segment Displays
- 4 X 3 Keypad
- 8 LEDs
- DC Motor
- 8 Switches
- Digital-to-Analogue Converter (DAC) - displayed on oscilloscope

Why EdSim51's Simulator and not some of the many other simulators that are available?

Many of the simulators for the 8051 that you will find are industry-standard. They are used by professional 8051-based embedded systems designers. While they show the state of the registers, memory and the port pins while code is being debugged, they do not have graphical representations of peripherals that can be used interactively to communicate with the 8051. EdSim51 have filled that need.

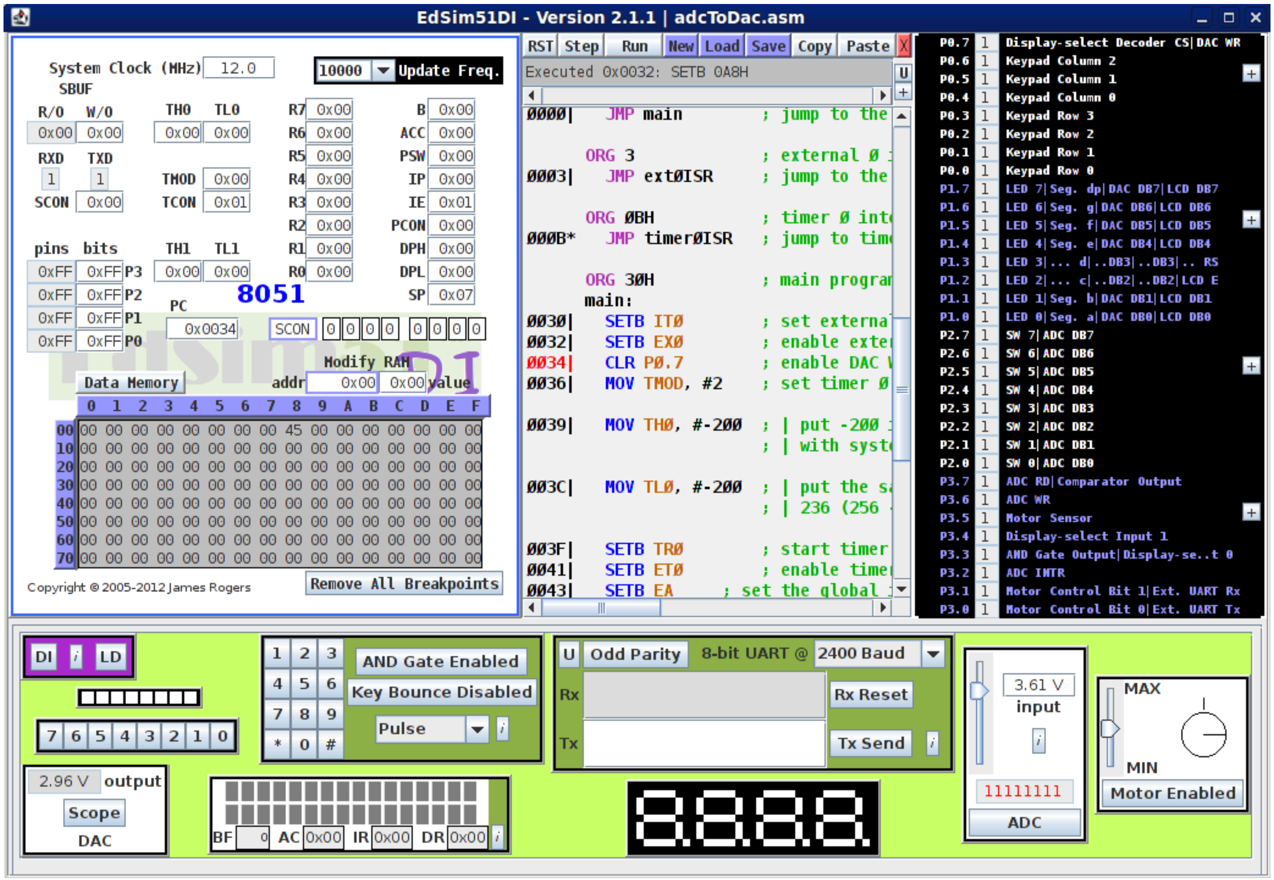
The student can learn how to scan a keypad, multiplex 7-segment displays, control a motor and count its revolutions, etc.

How to set up the 8051 Simulator:

1) Make sure you have Java installed, you can download it at http://java.com/en/download/index.jsp
2) Run the installer you downloaded
   a. Make sure to uncheck installing the Ask toolbar
3) Download http://www.edsim51.com/8051simulator/edsim51di.zip
   a. If the software has been updated since this writing, the newer version can be downloaded at http://www.edsim51.com/

4) Unzip the file

5) Inside the edsim51di folder, run edsim51di.jar



- In the middle window are your main controls where you will be able to write and execute your code
- The bottom window allows you to manipulate virtual inputs to the 8051, and see outputs on its various LCDs and LEDs
- On the left is a representation of what is stored in the memory and registers of your 8051
  ○ Note: To clear some of these requires closing and re-opening the simulator
- In the black box on the right you'll find a reference for how to access various "hardware" elements of the simulated 8051, such as switches and LEDs, and whether each currently sees a logic 1 or logic 0

Example 1:

This program displays the binary pattern from 0 to 255 (and back to 0) on the LEDs interfaced with port 1. A 1 in the pattern is represented by the LED on, while a 0 in the pattern is represented by the LED off.
However, logic 0 on a port 1 pin turns on the LED, therefore it is necessary to write the inverse of the pattern to the LEDs. The easiest way to do this is to send the data FFH to 0 (and back to FFH) to the LEDs Since port 1 is initially at FFH all we need to do is continuously decrement port 1.

```
start: DEC P1 ;decrement port 1
       JMP start ;and repeat
```

When running this program, best viewed with Update Freq. set to 1.

Example 2:

This program very simply echoes the switches on P2 to the LEDs on P1. When a switch is closed a logic 0 appears on that P2 pin, which is then copied to that P1 bit which turns on that LED. Therefore, a closed switch is seen as a lit LED and vice versa.

```
start:    MOV P1,P2    ;moved data on P2 pins to P1
          JMP start    ;and
```

r e
e a
p t

When running this program, best viewed with Update Freq. set to 1.

A Few Notes on the Assembler:

The 2-pass assembler with the EdSim51 Simulator is not a full-blown assembler. It does not link multiple files and only some of the directives you might expect are implemented. However, I feel it is adequate for the beginner. Below is a list of its features:

- All of the 8051 instructions are implemented, except for MOVX instructions, as the simulator does not handle external memory.
- JMP rel equates to either SJMP rel or AJMP rel. LJMP rel must be programmed explicitly.
- Similarly, CALL equates to ACALL. LCALL must be programmed explicitly.
- SET and EQU directives are implemented.

- ORG is implemented.
- USING directive (states which register bank is being used) is implemented.
- ARn equates to the register address, as specified by USING (if the register bank is not specified prior to ARn's use, register bank 0 is assumed).
- SFR names and SFR bit names equate to the appropriate address.
- HIGH followed by an operand in brackets equates to the high byte of the operand.
- LOW followed by an operand in brackets equates to the low byte of the operand.
- Labels are followed by a colon.
- The default for numerical values is decimal. Hex values can be entered by appending H after the number, or placing 0x before it. If H is used, the number cannot begin with a letter (example: F5H must be written as 0F5H). Binary values are entered by appending B after the number (as shown in the image below).
- The assembler is not case-sensitive.

Lab Tasks:

1. Show the output of the examples given in the manual.

Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:

Submit the Lab Report by writing all the examples and tasks which must include the following:

1. Brief description about how its work (3-5 lines)
2. The code
3. The results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.